Assignment

Chapter 3: Threads and Kernel Architecture

- **4.5** If a process exits and there are still threads of that process running, will they continue to run?

Assignment

Chapter 4:
Concurrency: Mutual Exclusion and Synchronization

- **1.** A student maioring in anthropoloay and minoring in computer science has embarked on a research project to see if African baboons can be taught about deadlocks. He locates a deep canyon and fastens a rope across it, so the baboons can cross handover-hand. Several baboons can cross at the same time, provided that they are all going in the same direction. If eastward moving and westward moving baboons ever get onto the rope at the same time, a deadock will result(the baboons will get stuck in the midde) because it is impossible for one baboon to climb over another one while suspended over the canyon.

Chapter 4

- If a baboon wants to cross the canyon, he must check to see that no other baboon is currently crossing in the opposite direction.Write a program using semaphores that avoids deadlock. Do not worry about a series of eastward moving baboons holding up the westward moving baboons indefinitely.

Assignment

Chapter 5:
Concurrency: Deadlock and Starvation

- **6.5(sixth Edition)** Consider the following snapshot of a system. There are no outstanding unsatisfied requests for resources.

available

r1	r2	r3	r4
2	1	0	0

	current allocation				maximum demand				still needs			
process	r1	r2	r3	r4	r1	r2	r3	r4	r1	r2	r3	r4
p1	0	0	1	2	0	0	1	2				
p2	2	0	0	0	2	7	5	0				
p3	0	0	3	4	6	6	5	6				
p4	2	3	5	4	4	3	5	6				
p5	0	3	3	2	0	6	5	2				

Chapter 5

- a. Compute what each process still might request and display in the columns labeled“still needs”.
- b. Is this system currently in a safe or unsafe state? Why?
- c. Is this system currently deadlocked? Why or why not?
- d. Which processes, if any, are or may become deadlocked?
- e. If a request from p3 arrives for(0, 1, 0, 0), can that request be safely granted immediately? In what state(deadlocked, safe, unsafe)would immediately granting that whole request leave the system? Which processes, if any, are or may become deadlocked if this whole request is granted immediately?

Chapter 5

- **6.6(sixth Edition)** : Apply the deadlock detection algorithm of Section 6.4 to the following data and show the results.

Available= (2 1 0 0)

Request = $\begin{pmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{pmatrix}$ Allocaiton = $\begin{pmatrix} 0 & 0 & 1 & 1 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{pmatrix}$